

Nextflow

Héctor Jesús López-Ordaz

2026-04-30

Tabla de contenidos

Planteamiento	1
Objetivo general	2
Objetivos específicos	2
Metodología	2
Nextflow script	2
Ejecución de Multiqc	5
Discusión	5
Conclusión	7

Planteamiento

El análisis de datos genómicos de secuenciación masiva requiere la ejecución de múltiples pasos computacionales; Uno de los primeros es aplicar el control de calidad y el recorte de secuencias. Tradicionalmente, estas tareas se han abordado mediante la ejecución manual de comandos o el desarrollo de *scripts* de Bash iterativos. Sin embargo, el enfoque clásico suele ser tedioso, ya que se debe analizar cada archivo de secuencias uno a uno por medio de `fastqc` o de manera conjunta encadenando con `multiqc`, por lo cual la escalabilidad es complicada aún más cuando los datos de secuenciación se han vuelto más económicos lo que incrementa el volumen de las muestras, creando un cuello de botella por el tiempo empleado.

Una de las herramientas, que propone resolver este problema es Nextflow. Surge como un sistema de gestión de flujos de trabajo (*workflows*) diseñado para solventar estas problemáticas, permitiendo la creación de *pipelines* de análisis de datos reproducibles, paralelizables y portables. En este trabajo, se explora la implementación de un flujo de trabajo en Nextflow para automatizar el uso de `fastqc` para generar reportes de las lecturas crudas, seguido de un *trimming* por `fastp` y nuevamente generar

reportes fastqc sobre los archivos procesados todo orquestado por un solo *script* de Nextflow. #
Objetivos

Objetivo general

Evaluar la eficiencia, reproducibilidad y escalabilidad de la implementación de un flujo de trabajo automatizado para el preprocesamiento de datos de secuenciación utilizando Nextflow, contrastando su arquitectura y ejecución frente a una aproximación de análisis secuencial y manual.

Objetivos específicos

- Desarrollar un *script* en Nextflow que estructure y automatice los procesos de control de calidad crudo (FastQC), recorte y filtrado de lecturas pareadas (fastp), y evaluación de calidad post-filtrado.
- Implementar el manejo concurrente de múltiples muestras mediante el uso de canales (*channels*) y aplicar un diseño modular reutilizando procesos a través de subflujos de trabajo (*subworkflows*).
- Comparar operativamente el flujo de trabajo construido en Nextflow frente a un análisis manual tradicional, evaluando las ventajas y desventajas de un *pipeline* automatizado en términos de escritura, tolerancia a fallos y trazabilidad.

Metodología

Nextflow script

Para iniciar, se reacomodó la carpeta de transcriptómica para tener un orden sobre los distintos proyectos, por lo cual a nivel transcriptómica (/home/hectorjl/4to/transcriptómica):

```
mkdir DE_analysis
mv * DE_analysis
mkdir nextflow_examples/{src,data,results}
mkdir nextflow_examples/results/multiqc
```

Moviéndonos entonces a la raíz de este proyecto:

```
cd nextflow_examples
```

Con esto podemos ahora trabajar en el script de Nextflow: Con el uso de: `vim /src/fastpc.nf`

```

// Define main parameters
params.fqs_dir = ""
params.qc_out = ""
params.fastp_dir = ""
params.cores = "2"

// Define the process to execute fastqc
process fastqc {
    // Output files of the process will be moved in "params.fastp_dir"
    // due to file size
    publishDir params.qc_out, mode: 'move'

    // Arguments expected by the process
    input:
    tuple val(SRR_name), path(fastq_path)
    val cores
    val phase
    // Output expected by the process
    output:
    path "${SRR_name}_${phase}_fqc"

    // fastqc run
    script:
    """
    mkdir -p ${SRR_name}_${phase}_fqc
    /export/apps/bioconda/bin/fastqc ${fastq_path} -o ${SRR_name}_${phase}_fqc -t ${cores}
    """
}

process trim_by_fastp {
    // Output of the process will be moved due to file size
    publishDir params.fastp_dir, mode: 'move'

    // Arguments expected
    input:
    tuple val(SRR_name), path(fastq_path)
    val cores

    // Output expected
    output:
    tuple val(SRR_name), path("clean_*.fastq")

    // fastp run
    script:
    """
    /export/apps/bioconda/envs/fastp/bin/fastp -i ${fastq_path[0]} -I ${fastq_path[1]} \
    -o clean_${SRR_name}_1.fastq -O clean_${SRR_name}_2.fastq \
    --cut_front --cut_front_window_size 4 --cut_front_mean_quality 20 \
    --cut_tail --cut_tail_window_size 4 --cut_tail_mean_quality 20 \
    --detect_adapter_for_pe --trim_poly_g -l 50 -w ${cores}
    """
}

// As process can not called twice in the same workflow

```

```

// to reuse processes it is need it to split into
// subworkflows

workflow raw_qc {
    // Argument expected
    take:
    SRR_pairs

    main:
    // Calls fastqc process, it sends three arguments
    // 1st -> The SRR pairs (read_1 and read_2)
    // 2nd -> Number of cores
    // 3rd -> A string that will be used to output name ("raw")
    fastqc(SRR_pairs, params.cores, "raw")
}

workflow clean_qc {
    // Argument expected
    take:
    SRR_pairs

    main:
    // Calls fastqc process, it sends three arguments
    // 1st -> The SRR pairs (read_1 and read_2)
    // 2nd -> Number of cores
    // 3rd -> A string that will be used to output name ("cleaned")
    fastqc(SRR_pairs, params.cores, "cleaned")
}

// Main workflow
workflow {
    main:
    // Define the files pairs using REGEX-like expression
    SRR_pairs = channel.fromFilePairs("${params.fqs_dir}/*_{1,2}.fastq")
    // These two processes are run in parallel
    // Call to the raw_qc workflow
    raw_qc(SRR_pairs)
    // Trim at the same time the fastp files
    trimmed = trim_by_fastp(SRR_pairs, params.cores)

    // After the trimming is done run fastqc in those files
    clean_qc(trimmed)
}

```

Ejecución del script:

```

nextflow run fastpc.nf \
--fqs_dir ../../DE_analysis/data/SRRs \
--qc_out ../results/fastqc \
--fastp_dir ../results/trimmed \
--cores 20

```

Realizando un tree de results/trimmed

```
results/trimmed
clean_SRR9126859_1.fastq
clean_SRR9126859_2.fastq
clean_SRR9126934_1.fastq
clean_SRR9126934_2.fastq
clean_SRR9127063_1.fastq
clean_SRR9127063_2.fastq
clean_SRR9127293_1.fastq
clean_SRR9127293_2.fastq
clean_SRR9127308_1.fastq
clean_SRR9127308_2.fastq
clean_SRR9127357_1.fastq
clean_SRR9127357_2.fastq
clean_SRR9127396_1.fastq
clean_SRR9127396_2.fastq
```

Ejecución de Multiqc

Ahora hagamos un multiqc de los archivos fastqc generados.

```
multiqc results/fastqc/*/* -o results/multiqc
```

Discusión

Comparemos con el multiqc que fue generado con los métodos más tradicionales y el generado por el *pipeline*.

1. Heatmap de archivos crudos:

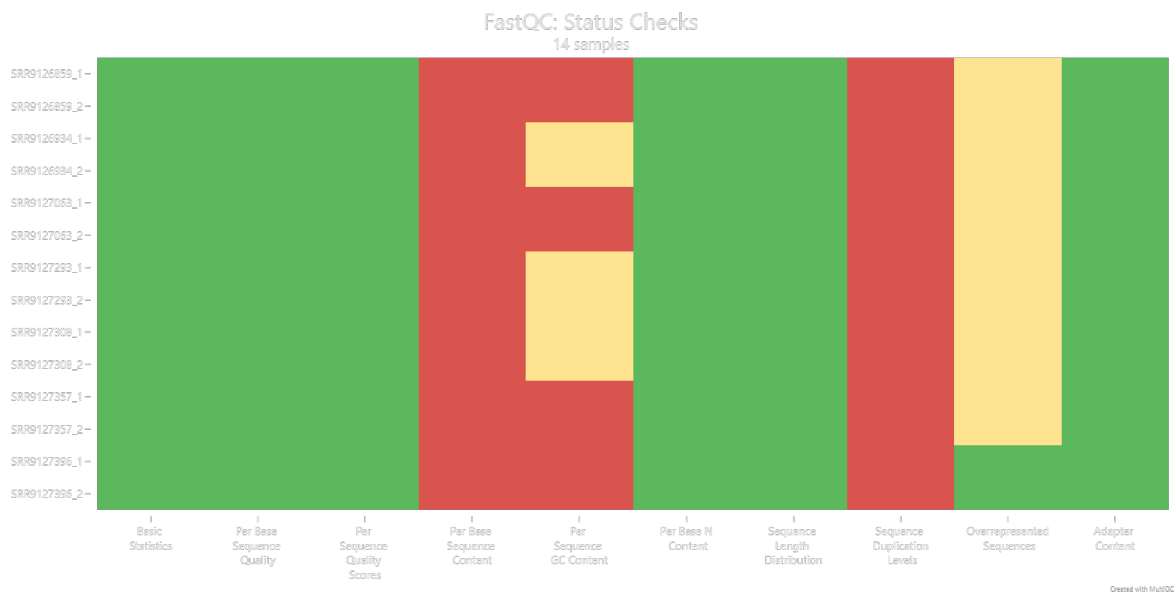


Figura 1: Raw Status Heatmap

2. Heatmap de archivos limpios, hecho con el *pipeline* creado en Nextflow:

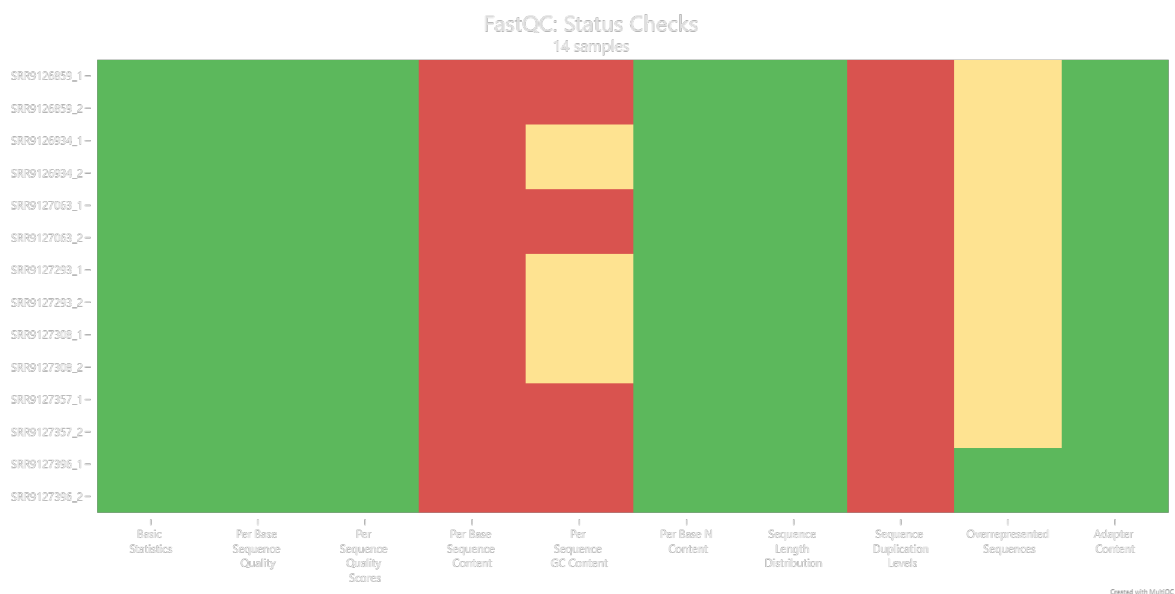


Figura 2: Nf Heatmap Status

3. Heatmap de archivos limpios, hecho de manera tradicional:

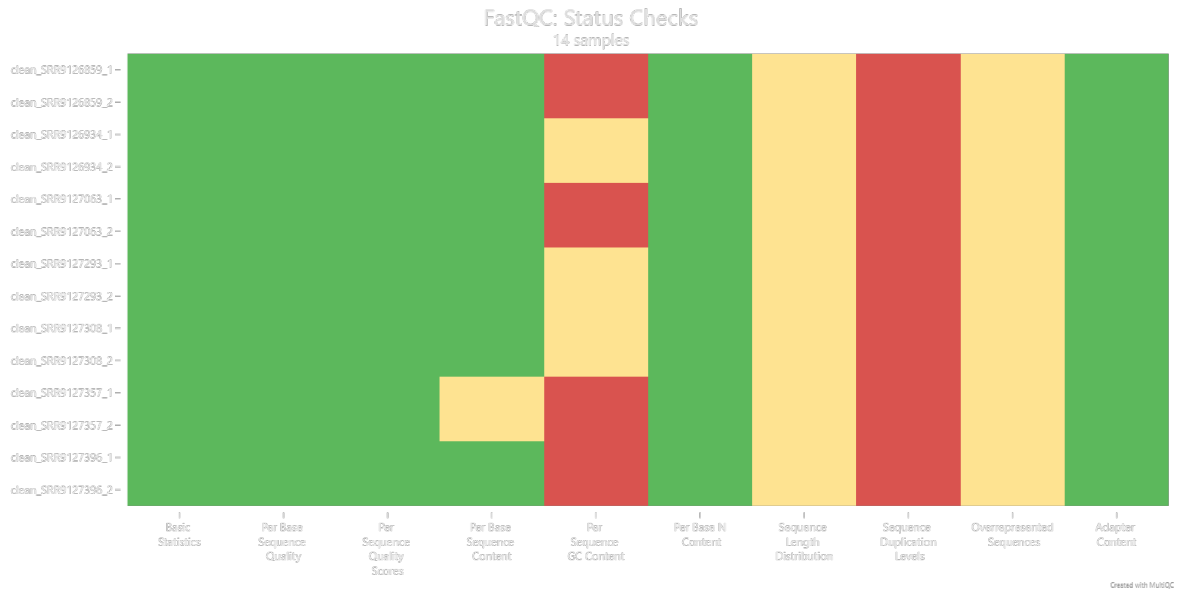


Figura 3: Clean Heatmap Status

Al inspeccionar los gráficos de control de calidad, se observa que la diferencia visual entre el *heatmap* de las lecturas crudas y el de las lecturas procesadas por el *pipeline* de Nextflow es mínima. En contraste, el *trimming* ejecutado de manera tradicional muestra una mejora sustancial y evidente en la calidad de las secuencias.

La gran ventaja de Nextflow es a su vez su principal riesgo si no se invierte el tiempo necesario en la optimización del código. Es posible alcanzar un nivel de calidad idéntico al de un experto humano mediante un *pipeline*, pero esto exige construir un flujo de trabajo mucho más robusto, parametrizado cuidadosamente y, en ocasiones, con algoritmos de toma de decisiones dinámicas incluidos en el propio *script*.

Conclusión

La implementación de Nextflow para el preprocesamiento de datos genómicos representa un avance significativo en la resolución de los cuellos de botella computacionales de la secuenciación masiva. El *pipeline* desarrollado en este trabajo cumplió satisfactoriamente con la automatización del control de calidad y el recorte de adaptadores, demostrando una superioridad indudable en términos de escalabilidad, procesamiento concurrente y reproducibilidad frente a los *scripts* secuenciales de Bash.

No obstante, el análisis comparativo subraya que la automatización no exime de la necesidad de rigor analítico y curación biológica. Las discrepancias observadas en la calidad del *trimming* evidencian que un flujo de trabajo es tan efectivo como los parámetros que lo rigen. La transición hacia herramientas

de orquestación como Nextflow es un estándar necesario en la bioinformática moderna, sin embargo, para que estos *pipelines* sean verdaderamente útiles, deben someterse a una etapa intensiva de diseño y calibración. En última instancia, la eficiencia operativa de Nextflow debe estar siempre respaldada por el criterio humano para garantizar que la calidad biológica de los datos no se vea comprometida por la velocidad de la automatización.